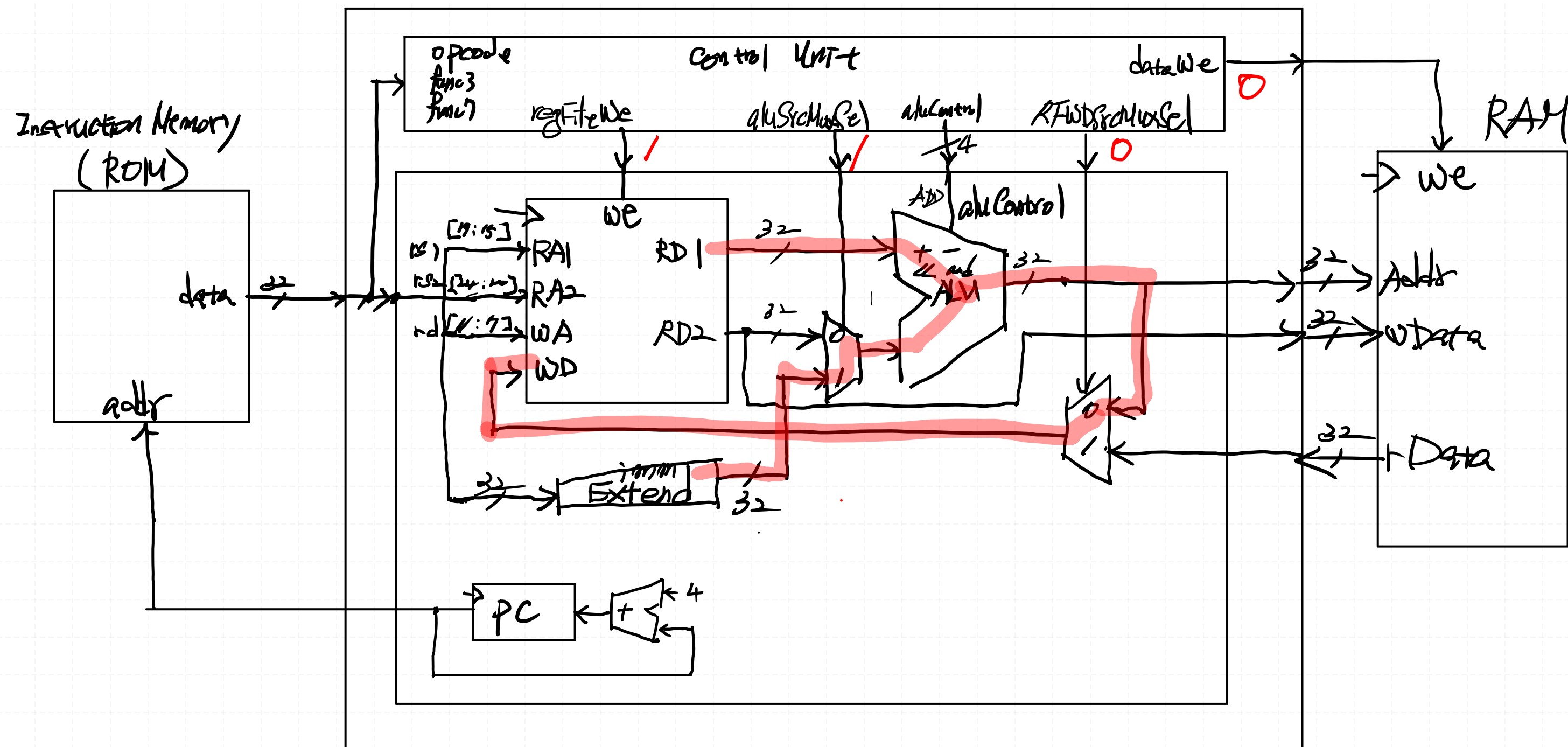
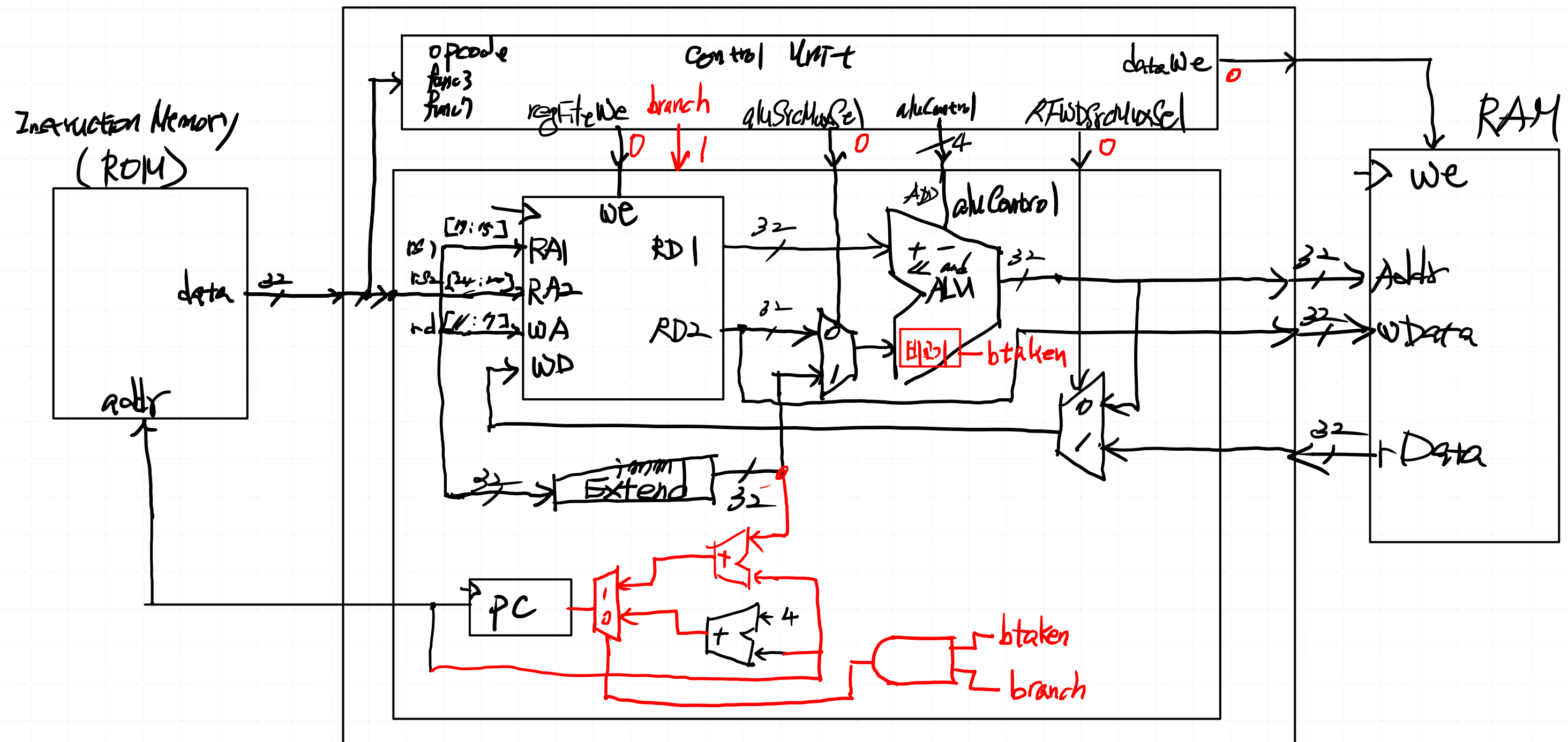




31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	TYPE	MNEMONIC	NAME	Descript
immediate (7)							shift amount (5)					Regitster Source 1 (5)					Funct3 (3)			Register Destination (5)					Opcode (7)										
imm[11:0]												rs1					0 0 0			rd					0 0 1 0 0 1 1							I-TYPE	ADDI	ADD Immediate	rd = rs1 + imm
imm[11:0]												rs1					0 1 0			rd					0 0 1 0 0 1 1								SLTI	Set Less Than Imm	rd = (rs1 < imm) ? 1 : 0
imm[11:0]												rs1					0 1 1			rd					0 0 1 0 0 1 1								SLTIU	Set Less Than Imm (U)	rd = (rs1 < imm) ? 1 : 0
imm[11:0]												rs1					1 0 0			rd					0 0 1 0 0 1 1								XORI	XOR Immediate	rd = rs1 ^ imm
imm[11:0]												rs1					1 1 0			rd					0 0 1 0 0 1 1								ORI	OR Immediate	rd = rs1 imm
imm[11:0]												rs1					1 1 1			rd					0 0 1 0 0 1 1								ANDI	AND Immediate	rd = rs1 & imm
0	0	0	0	0	0	0	shamt					rs1					0 0 1			rd					0 0 1 0 0 1 1								SLLI	Shift Left Logical Imm	rd = rs1 << shamt[0:4]
0	0	0	0	0	0	0	shamt					rs1					1 0 1			rd					0 0 1 0 0 1 1								SRLI	Shift Right Logical Imm	rd = rs1 >> shamt[0:4]
0	1	0	0	0	0	0	shamt					rs1					1 0 1			rd					0 0 1 0 0 1 1								SRAI	Shift Right Arith Imm	rd = rs1 >>> shamt[0:4]



if, for while switch $PC = PC + imm$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	TYPE	MNEMONIC	NAME	Descript
immediate (7)							Regitster Source 2 (5)					Regitster Source 1 (5)					Funct3 (3)			immediate (5)					Opcode (7)							B-TYPE	BEQ	Branch ==	if(rs1 == rs2) PC += imm
imm[12][10:5]							rs2					rs1					0 0 0			imm[4:1][11]					1 1 0 0 0 1 1								BNE	Branch !=	if(rs1 != rs2) PC += imm
imm[12][10:5]							rs2					rs1					1 0 0			imm[4:1][11]					1 1 0 0 0 1 1								BLT	Branch <	if(rs1 < rs2) PC += imm
imm[12][10:5]							rs2					rs1					1 0 1			imm[4:1][11]					1 1 0 0 0 1 1								BGE	Branch ≥	if(rs1 ≥ rs2) PC += imm
imm[12][10:5]							rs2					rs1					1 1 0			imm[4:1][11]					1 1 0 0 0 1 1								BLTU	Branch < (U)	if(rs1 < rs2) PC += imm
imm[12][10:5]							rs2					rs1					1 1 1			imm[4:1][11]					1 1 0 0 0 1 1								BGEU	Branch ≥ (U)	if(rs1 ≥ rs2) PC += imm
imm[12][10:5]							rs2					rs1					1 1 1			imm[4:1][11]					1 1 0 0 0 1 1										

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	TYPE	
11 10 9 8 7 6 5 4 3 2 1 0 imm[11:0] (7)							4 3 2 1 0 shift amount (5)					Register Source 1 (5)					Funct3 (3)			Register Destination (5)					Opcode (7)								
imm[11:0]							rs1					0 0 0			rd					0 0 0 0 0 1 1							L-TYPE						
imm[11:0]							rs1					0 0 1			rd					0 0 0 0 0 1 1													
imm[11:0]							rs1					0 1 0			rd					0 0 0 0 0 1 1													
imm[11:0]							rs1					1 0 0			rd					0 0 0 0 0 1 1													
imm[11:0]							rs1					1 0 1			rd					0 0 0 0 0 1 1													
imm[11:0]							rs1					0 0 0			rd					0 0 1 0 0 1 1							I-TYPE						
imm[11:0]							rs1					0 1 0			rd					0 0 1 0 0 1 1													
imm[11:0]							rs1					0 1 1			rd					0 0 1 0 0 1 1													
imm[11:0]							rs1					1 0 0			rd					0 0 1 0 0 1 1													
imm[11:0]							rs1					1 1 0			rd					0 0 1 0 0 1 1													
imm[11:0]							rs1					1 1 1			rd					0 0 1 0 0 1 1													
0 0 0 0 0 0 0							shamt					rs1			rd					0 0 1 0 0 1 1													
0 0 0 0 0 0 0							shamt					rs1			rd					0 0 1 0 0 1 1													
0 1 0 0 0 0 0							shamt					rs1			rd					0 0 1 0 0 1 1													

Imm의 고정 bit를 최대한 똑같은 위치에 맞춘다.

L, I, S, B, J, JL Type에서 최대한 같은 위치의 Imm bit 위치를 같이 한다.
H/W 최적화이 도움이 된다.

B, J Type의 경우 Imm 값이 ROM(PC) 주소의 offset 값이 된다.

ROM의 주소는 항상 정수가 되므로 Imm[0]은 항상 '0' 값이 된다.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	TYPE
11 10 9 8 7 6 5 imm[11:5]							Register Source 2 (5)					Register Source 1 (5)					Funct3 (3)			4 3 2 1 imm[4:0]				0	Opcode (7)							
imm[11:5]							rs2					rs1					0 0 0			imm[4:0]					0 1 0 0 0 1 1							S-TYPE
imm[11:5]							rs2					rs1					0 0 1			imm[4:0]					0 1 0 0 0 1 1							
imm[11:5]							rs2					rs1					0 1 0			imm[4:0]					0 1 0 0 0 1 1							

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	TYPE
11098765 immediate (7)							Register Source 2 (5)					Register Source 1 (5)					Funct3 (3)			4321 immediate (5)				11 Opcode (7)								
imm[12][10:5]							rs2					rs1					0 0 0			imm[4:1][11]				1 1 0 0 0 1 1							B-TYPE	
imm[12][10:5]							rs2					rs1					0 0 1			imm[4:1][11]				1 1 0 0 0 1 1								
imm[12][10:5]							rs2					rs1					1 0 0			imm[4:1][11]				1 1 0 0 0 1 1								
imm[12][10:5]							rs2					rs1					1 0 1			imm[4:1][11]				1 1 0 0 0 1 1								
imm[12][10:5]							rs2					rs1					1 1 0			imm[4:1][11]				1 1 0 0 0 1 1								
imm[12][10:5]							rs2					rs1					1 1 1			imm[4:1][11]				1 1 0 0 0 1 1								

그러므로 B, J Type에서 Imm[0]이 없이 감위로 '0'을 넣어주어도 된다.

그 효과는 offset 범위값을 넓힐 수 있다.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	TYPE
imm[31:12] (20)																				Register Destination (5)					Opcode (7)							
imm[31:12]																				rd					0 1 1 0 1 1 1							LU-TYPE
imm[31:12]																				rd					0 0 1 0 1 1 1							AU-TYPE
imm[20:10] (11)																				rd					1 1 0 1 1 1 1							J-TYPE
imm[11:0] (12)																				rs1					1 1 0 0 1 1 1							JL-TYPE

※ 명령어가 입력된 ROM의 주소 특징은 4의 배수이다.
Imm 2bit 고정. 최대값 주소범위를 넓히고 싶다.

B-Type (Branch) 명령어에서 Immediate 필드의 비트들이 연속적으로 나열되지 않는 이유는 하드웨어 구현과 분기 오프셋 계산의 효율성을 위해 설계된 포맷 때문입니다. 구체적인 이유를 살펴보겠습니다.

1. Immediate 비트들이 재배열되는 이유

B-Type 명령어에서 즉시값(Immediate)은 **점프 오프셋(branch offset)**을 나타내는데, 실제로는 12비트 크기의 signed offset을 가지며, 해당 비트들은 다음과 같이 분할되어 저장됩니다.

즉시값 비트	위치
imm[12]	비트 31
imm[10:5]	비트 30-25
rs2	비트 24-20
rs1	비트 19-15
funct3	비트 14-12
imm[4:1]	비트 11-8
imm[11]	비트 7
opcode	비트 6-0

즉, Immediate 값의 특정 비트들이 서로 떨어져 있는 이유는 RISC-V 명령어 포맷에서 효율적인 디코딩을 위해 일부 필드를 분리했기 때문입니다.

2. 하드웨어 구현 관점

하드웨어에서 B-Type 명령어를 디코딩할 때, 즉시값(Immediate) 비트들이 특정 위치에 배치되면 간단한 배선 조작만으로 오프셋을 복원할 수 있습니다. 즉, 명령어를 해석하는 하드웨어 로직을 단순하게 만들기 위한 설계입니다.

즉시값을 하드웨어가 최적의 방식으로 처리할 수 있도록 아래처럼 조합합니다:

$$Immediate = \text{Sign-Extend}(\text{imm}[12], \text{imm}[11], \text{imm}[10:5], \text{imm}[4:1], 0)$$

여기서 마지막 0은 B-Type 명령어가 항상 2바이트 정렬된 주소로 점프하기 때문에 추가됩니다. 즉, 오프셋은 항상 2의 배수이므로 최하위 비트는 0이어야 합니다.

3. 정렬되지 않은 Immediate 비트의 장점

Immediate 비트들이 이렇게 나누어져 배치되면 다음과 같은 장점이 있습니다:

- 하드웨어 설계 간소화: 즉시값을 추출하고 확장하는 로직을 단순하게 유지할 수 있습니다.
- 분기 대상 주소의 계산 최적화: B-Type 명령어는 상대 주소(offset) 방식으로 동작하는데, 비트 배열을 이렇게 배치하면 주소 계산 시 쉬프트(Shift) 연산을 최소화할 수 있습니다.
- 명령어 포맷의 일관성 유지: RISC-V는 여러 유형의 명령어 (R-Type, I-Type, S-Type, B-Type 등)를 표준화된 구조로 설계했기 때문에, Immediate 비트 배열도 특정 규칙에 따라 배치되었습니다.

4. 다른 명령어와의 비교

- I-Type (예: ADDI)
 - Immediate 값이 **연속적인 12비트 (imm[11:0])**로 배치됨
- S-Type (예: SW, SB, SH)
 - Immediate 값이 imm[11:5]와 imm[4:0]로 나누어짐
- B-Type (분기 명령어)
 - Immediate 값이 imm[12], imm[10:5], imm[4:1], imm[11]로 배치됨
 - 여기서 imm[11]이 bit 7에 위치한 이유는 하드웨어 최적화를 위해 점프 대상 주소 계산을 쉽게 하기 위함

5. 결론

Immediate 값의 비트 순서가 연속적이지 않은 이유는 하드웨어에서 효율적으로 디코딩하고, 점프 오프셋을 빠르게 계산할 수 있도록 설계되었기 때문입니다. 이를 통해 분기 명령어를 실행할 때 필요한 연산이 줄어들고, 명령어 디코딩이 최적화됩니다.

즉, B-Type 명령어의 Immediate 비트 순서는 명령어 포맷의 일관성과 하드웨어 효율성을 고려한 설계적 결정입니다.